

Assembleia 06 — Mapas Avançados

Pedido do Tiago, verbatim: mapas externos "no estilo dos jogos Kingdom Hearts", interiores detalhados o bastante pra diferenciar móveis, névoa de guerra sobre áreas inexploradas com marcador de missão visível antes de revelar o resto, mapa só disponível dentro da cidade (bloqueado em masmorra/missão distante até sair), e desbloqueio de mapa + missões ao conversar com NPC na chegada numa área nova.

Ele mesmo avisou: "isso é bem mais denso do que parece" — concordo, e a consulta individual confirma. Método completo, os 4 pontos bloqueantes enfrentados de frente antes dos finalistas.

1. Baseline — os 4 pontos difíceis, resolvidos aqui antes de votar

1.1 Direito autoral — "estilo Kingdom Hearts" sem usar nada da Square Enix

"Estilo Kingdom Hearts" aplicado ao mapa do mundo é uma **composição** reconhecível — mapa ilustrado, colorido, com locais como nós ligados por caminhos sinuosos — não uma obra protegida por si só. Esse padrão compositivo aparece em dezenas de jogos antes e depois de Kingdom Hearts (mapas de mundo de RPG são um gênero visual, não uma invenção da Square Enix). O que **não pode**, sem exceção: o símbolo de coração dos mundos, nomes de mundos Disney/Final Fantasy, silhuetas de personagens, paleta/tipografia copiada 1:1 do jogo real, ou qualquer asset extraído do jogo.

Como atingir o estilo dentro da disciplina do acervo (CC0/domínio público/gerado por código, com proveniência registrada — igual sempre):

- **Terreno:** em vez do preenchimento em cor sólida quase lisa que usamos hoje (docs/MAPAS-PROVENIENCIA.md), gerar textura por código (gradiente + ruído Perlin/simplex via PIL, parametrizado por bioma — grama, água, pedra) pra um efeito mais pintado/orgânico, sem depender de sprite nenhum de terceiro pra essa camada.
- **Caminhos:** curvas suaves (Bézier) desenhadas por código conectando os locais, em vez da grade reta atual — é o elemento mais reconhecível do "estilo mapa de mundo estilizado" e é 100% geometria nossa, sem risco de direito autoral nenhum.
- **Cenário/decoração:** sprites CC0 do Kenney (mais variedade que hoje — ver 1.1.1) pontuando o mapa, não cobrindo ele inteiro.
- **Expectativa realista, dita sem rodeio:** isso aproxima a *sensação* de "mapa de mundo ilustrado colorido", não replica qualidade de ilustração feita à mão por um estúdio AAA. É uma melhora real sobre o que existe, não uma promessa de indistinguível do jogo de referência — ninguém deveria esperar isso de um pipeline automatizado.

1.1.1 — A arte CC0 do Kenney já usada aguenta o nível de detalhe pedido (mesa, cadeira, escada, balcão)?

Não, o pacote "RPG Base" que já usamos não tem isso — pesquisado agora, não de memória: aquele pacote (paredes, portas, árvores, prateleira, barril) não tem tiles de mobília individualizada. **Existe outra fonte CC0 do próprio Kenney que cobre exatamente essa necessidade**, confirmada por pesquisa ao vivo hoje:

Pacote	Licença	Conteúdo relevante	Fonte
Roguelike/RPG pack	CC0	~1.700 tiles de 16×16px, tags incluem "furniture" e "town" — estilo pixel top-down, mesma família visual do que já usamos	kenney.nl/assets/roguelike-rpg-pack
Furniture Kit	CC0	120+ objetos (mesas, cadeiras, estantes, cozinha, banheiro), renders top-down E isométricos disponíveis	kenney.nl/assets/furniture-kit

Recomendação: usar o **Roguelike/RPG pack** como fonte primária — mesma família de pixel art do "RPG Base" já em produção (consistência visual entre os dois), com furniture nativa. O "Furniture Kit" fica como fonte secundária só se faltar algum móvel específico (é um estilo de render mais realista, meio passo fora da família visual atual — usar com moderação, não como base). **Antes de produção:** gerar um contact sheet dos tiles candidatos em 2-3x e inspecionar visualmente — mesma disciplina que pegou o problema de autotile do "RPG Base" da vez passada (docs/MAPAS-PROVENIENCIA.md, achado real) — não confiar na miniatura pequena da página do Kenney.

1.2 Névoa de descoberta — onde persiste, como o marcador de missão aparece sem revelar o resto

Escopo de persistência: por **campanha** (campaign_sessions), não por personagem nem por conta — é a mesma unidade que já guarda messages/history/turn_count/seed (Assembleia 05). Faz sentido porque "o que já foi descoberto" é progresso *daquela história*, não um dado permanente do personagem (se o personagem sobreviver a uma nova campanha depois, começa a explorar de novo — coerente com o próprio recomeço variado já aprovado).

Dois camadas de dado independentes (decisão de design central desta seção, evita confundir "terreno visível" com "ponto de interesse conhecido"):

- 1 discovered — lista de ids de locais/regiões que o jogador já visitou fisicamente. Controla o que aparece "revelado" no mapa.
- 2 knownMarkers — lista de ids de pontos de interesse conhecidos *por informação*, mesmo sem visita física (uma missão contada por um NPC aponta pra lá). Controla quais marcadores aparecem *por cima* da névoa, mesmo em área ainda escura.

Um marcador de missão em knownMarkers mas cujo local não está em discovered aparece sozinho, sem revelar o terreno ao redor — as duas listas nunca se misturam. Isso é exatamente o comportamento pedido: "as marcações das localizações das missões... são marcadas" sem expor o resto.

Custo em schema: duas colunas jsonb novas em campaign_sessions (discovered, known_markers) — aditivo, mesmo padrão de sempre (ALTER TABLE ... ADD COLUMN IF NOT EXISTS), sem tabela nova necessária pra v1.

Nota honesta sobre profundidade visual (ver Seção 4, é um dos eixos reais de divergência): existem dois jeitos de *renderizar* essa névoa — (a) **por nó**: local não descoberto simplesmente não aparece marcado no mapa (mais simples, mais barato, não é "escuridão" de verdade); (b) **overlay visual real**: uma camada semitransparente escura por cima do mapa, com "furos" nas áreas exploradas — é literalmente o que o Tiago descreveu ("o mapa fica escuro"), mas é tecnicamente mais pesado (compositar uma máscara em canvas por cima do Leaflet, sincronizada com pan/zoom — não existe precedente nenhum disso no código hoje). Os dois usam a mesma persistência de dado (`discovered/knownMarkers`); a diferença é só na camada de renderização. Tratado como eixo de decisão na votação, não resolvido aqui.

1.3 Regra de acesso ao mapa — quem decide "isto é uma masmorra"

O ponto mais difícil, resolvido com cuidado, como pedido.

Hoje o mestre narra livre e só é obrigado a escolher entre valores fixos já registrados no código quando toca em mapa (`moveTo` só aceita um dos 5 ids conhecidos; `enterInterior` só é válido pros 3 que têm interior desenhado) — o mestre **nunca inventa estrutura de mapa livremente**, só escolhe dentro do que o registro já define. A resposta certa pra "quem decide que isto é uma masmorra" segue o mesmo princípio, por consistência e por segurança (é literalmente o motivo do `SYSTEM_PROMPT` já listar exatamente quais locais têm interior — pra o mestre nunca inventar uma transição inválida):

Registro fixo, não o mestre livre. Cada local conhecido em `maps.js/data.js` ganha um campo tipo: `'cidade' | 'masmorra' | 'missao_distante'`, definido pelo Game System Designer/Narrative Writer com antecedência — o código consulta esse registro sempre que `partyAt` muda, e decide se o mapa fica disponível (`cidade`) ou bloqueado (`masmorra/missao_distante`) até o jogador sair. **Zero custo de token** — é consulta local, o mestre nem precisa saber que essa regra existe.

O que fazer com locais que a narração cria na hora, sem estarem no registro (uma "missão distante" específica que o mestre está narrando ali, não uma localização com mapa próprio desenhado)? Pra esses, o mestre sinaliza um campo novo e simples no `mapHint` — `"remoteArea": true | false | null` — "estou narrando uma cena fora do alcance do mapa da cidade agora, sim ou não". O código trata isso do mesmo jeito que trataria um local tipo `masmorra/missao_distante` do registro: mapa bloqueado até `remoteArea` voltar a `false` ou o jogador retornar a um `partyAt` conhecido. Custo: ~5-8 tokens por turno, só nos turnos em que o campo não é `null` (a maioria dos turnos em cidade nunca usa esse campo).

Essa combinação (registro fixo pro que já existe + sinal simples do mestre pro que é narrado ad-hoc) resolve o problema sem depender da consistência do LLM entre turnos pra classificar locais que **já conhecemos** — só pede o mínimo do mestre pro que é genuinamente improvisado.

1.4 Desbloqueio por NPC — precisa de um sistema de missões antes?

Resposta direta: um sistema de missões *completo* (objetivos, critérios de conclusão, recompensas, jornal/diário de missões na UI) — não, e não deveria ser construído agora, seria escopo muito maior que o pedido. Um sistema *mínimo* de "marcador de missão" — sim, é necessário, mas é pequeno:

```
{ id: string, titulo: string, localId: string, status: 'revelada' | 'concluida' }
```

Guardado como `jsonb` dentro de `campaign_sessions` (mesmo padrão de `discovered/known_markers`, sem tabela nova). O mestre, ao narrar uma conversa de chegada com um NPC que aponta uma missão, devolve um campo novo no `mapHint` — `"revealMission": { "id": "...", "titulo": "...",`

"localId": "..."} | null — o código adiciona esse id em knownMarkers (Seção 1.2) e cria a entrada em missions. **Isso não precisa de catálogo de missões pré-escrito pra funcionar** — o próprio mestre nomeia a missão na hora, dentro do enredo que já está narrando. Custo: ~30-50 tokens, só nos turnos (raros) em que uma missão é revelada — não em todo turno.

Tamanho real do corte: "marcador de missão" (o que a Seção 1.4 descreve) é pequeno — um campo novo no contrato JSON + uma lista jsonb. "Sistema de missões" de verdade (múltiplos objetivos por missão, condição de conclusão verificável, recompensa, tela de diário) é uma feature própria, do tamanho de uma assembleia inteira sozinha — não entra aqui.

2. Custo total quantificado

Item	Custo recorrente (por turno)	Custo único
Registro fixo de tipo de local (Seção 1.3)	0 — consulta local, mestre nem sabe que existe	—
mapHint.remoteArea	~5-8 tokens, só em turnos fora de local conhecido	—
mapHint.revealMission	~30-50 tokens, só em turnos que revelam missão (raro)	—
Instruções novas no SYSTEM_PROMPT (explicar os dois campos novos pro mestre)	~80-120 tokens, todo turno (o prompt inteiro é reenviado sempre)	—
Névoa de descoberta (persistência + renderização)	0 — puro client/banco, mestre não participa	—
Arte nova (terreno por código + Kenney Roguelike/RPG pack)	0 — produzida uma vez, servida como imagem estática	custo de produção (meu tempo), não de token

Total recorrente realista: ~80-130 tokens/turno adicionais na imensa maioria dos turnos (só o aumento do SYSTEM_PROMPT, que é fixo e sempre enviado) — em ~2.850 tokens/turno já estabelecidos (Assembleia 04), isso é **~3-4,5% de aumento por turno**, na mesma ordem de grandeza do custo da semente de campanha já aprovada (Assembleia 05, ~3%). Empilhado com a semente, o overhead combinado fica em torno de **6-8% acima da linha de base**, ainda longe de dobrar o consumo — mas é real, soma com o que já existe, e nenhum dos dois tem o mecanismo de proteção de orçamento (Assembleia 04) rodando ainda.

3. Consulta individual aos agentes

Game System Designer

O registro fixo de tipo de local (cidade/masmorra/missão distante) não mexe na fórmula D6 — é só dado de cena, trato como extensão de maps.js, não como mudança de regra. Prefiro decidir isso com registro fixo, não confiar no mestre classificar sozinho toda vez (consistência

flexibilidade aqui). Sobre missão: apoio a versão mínima (marcador),

não vejo necessidade de sistema completo agora — se um dia entrar recompensa/XP de missão, aí sim vira decisão minha de verdade.

Narrative Writer

O desbloqueio de missão ao conversar com NPC (item 3) é o ponto que mais me interessa — é o tipo de recompensa narrativa que dá peso a chegar num lugar novo. Ressalva real: cada "missão distante" com mapa próprio desenhado precisa da mesma curadoria que os 5 locais atuais tiveram — não dá pra ter locais novos infinitos sem meu trabalho de verdade em cada um. Prefiro **poucos locais distantes bem feitos** a muitos genéricos. A regra rígida de acesso (item 2) me interessa menos — é mais uma restrição de UX que uma alavanca narrativa; toparia ela entrar depois sem perda real pro jogador nesta fase.

Frontend Engineer

O maior risco técnico novo desta assembleia inteira é a névoa **visual real** (Seção 1.2b) — hoje não existe nenhuma composição de canvas por cima do Leaflet no código, seria a primeira vez. Prefiro entregar a versão por nó (Seção 1.2a) primeiro — reaproveita o padrão de marcador que já existe no MapPanel, sem risco novo — e trato o overlay escuro de verdade como melhoria visual de uma fase seguinte, depois de validar que o dado (`discovered/knownMarkers`) está sólido. A arte nova (interiores com mobília, terreno por código) é trabalho de composição offline (script Python), não toca em `components.jsx` além de trocar os PNGs — baixo risco.

Backend Engineer

Sem tabela nova pra v1 — `discovered`, `known_markers`, `missions` como três colunas `jsonb` aditivas em `campaign_sessions`, mesmo padrão já usado pro `seed` (Assembleia 05). Se missões crescerem de verdade depois (objetivos/recompensa), aí sim val a pena uma tabela própria — não antes de precisar.

AI Master Engineer

A conta da Seção 2 é a meu ver o dado mais importante desta assembleia: esse pacote sozinho fica em ~3-4,5% de overhead por turno — empilhado com a semente de campanha (Assembleia 05), o total combinado passa dos 6%, e **nenhum dos dois** tem o mecanismo de proteção de orçamento (Assembleia 04) rodando em produção ainda. Não sou contra a feature — sou a favor de manter o pacote **mínimo necessário** agora (`remoteArea` + `revealMission`, nada mais no prompt) e sinalizar, de novo, que abrir pra mais testers continua esperando aquele mecanismo, não importa quantas features boas se empilhem antes dele.

Infra Engineer

Assets novos (Kenney Roguelike/RPG pack, possivelmente Furniture Kit) somam no máximo algumas dezenas de KB — sem impacto de hosting. Colunas `jsonb` novas, sem custo de infra.

Code QA Engineer

Dois pontos pro checklist: (1) proveniência do Roguelike/RPG pack precisa entrar em `docs/MAPAS-PROVENIENCIA.md` antes de qualquer PNG novo ir pra produção — mesma disciplina de sempre; (2) o registro fixo de tipo de local (Seção 1.3) é exatamente o tipo de decisão que evita um problema de segurança/consistência real — reforço que o mestre **nunca** deveria poder inventar um tipo de local novo livremente, só usar `remoteArea` como sinal binário pro que não está no registro. Concordo com

a leitura da Seção 1.3 como está.

Test Engineer

Isso é candidato perfeito pra regra do fluxo encadeado: visitar cidade → entrar em local com interior → sair pra uma cena de missão distante (`remoteArea: true`) → conversar com NPC (revela missão) → voltar pra cidade → confirmar que o mapa reapareceu, o marcador da missão está lá, e o resto do mapa continua do jeito que estava. Prefiro validar isso numa fatia menor (Seção 1.2a, sem overlay visual) antes de somar mais uma camada de risco (overlay de canvas) no mesmo ciclo de teste.

Realtime Multiplayer Engineer

Sem posição forte — mas registrando pro futuro: `discovered/known_markers/missions` hoje ficam por campanha (1 jogador); se sala voltar, isso passa a ser estado **compartilhado do grupo**, não de um jogador individual — mesma nota que já registrei antes pra não esquecer de novo.

4. Síntese — convergência e divergência real

Convergência forte: registro fixo de tipo de local (não confiar no mestre classificar sozinho) — unânime. Marcador de missão mínimo, sem sistema completo — unânime. Duas camadas de dado separadas pra névoa (descoberto vs. conhecido) — unânime.

Três eixos reais de divergência, viram os finalistas:

- 1 **Profundidade da névoa:** por nó (Frontend, Test, AI Master — de risco/custo) vs. overlay visual real desde já (ninguém defendeu isso com prioridade máxima, mas é o que o Tiago descreveu literalmente — registrado como tensão real, não resolvida por consenso).
 - 2 **O que entra nesta rodada:** pacote mínimo só de arte (ninguém defendeu isso sozinho) vs. pacote com regra de acesso e névoa por nó (maioria) vs. pacote completo com marcador de missão incluso (Narrative Writer puxa por isso).
 - 3 **Urgência do orçamento:** AI Master Engineer mantém a mesma posição de cautela das Assembleias 04/05 — cada feature nova empilhada sem o mecanismo de proteção pronto é risco acumulado, não zerado.
-

5. Cinco finalistas

Finalista 1 — "Só arte"

Melhora visual dos mapas existentes (exteriores mais estilizados, interiores com mobília real) — sem névoa, sem regra de acesso, sem missão. Puramente cosmético sobre o sistema atual.

Finalista 2 — "Arte + névoa por nó, sem regra de acesso nem missão"

F1 + `discovered/known_markers` com renderização simples (local aparece/some do mapa, sem escurecimento visual) — mais barato e menos arriscado tecnicamente, mas não resolve os itens 2 e 3 do pedido.

Finalista 3 — "Arte + névoa por nó + regra de acesso, marcador de missão mínimo incluso"

F2 + registro fixo de tipo de local (`mapHint.remoteArea` pro que é ad-hoc) + `mapHint.revealMission` mínimo. Cobre os itens 1 (parcial — sem overlay visual), 2 e 3 do pedido original.

Finalista 4 — "Pacote completo, tudo nesta rodada"

F3 + overlay visual real de escuridão (Seção 1.2b) já nesta fase — o item 1 fica 100% fiel ao que foi pedido, mas soma o maior risco técnico novo (canvas sobre Leaflet, nunca feito antes) no mesmo ciclo que todo o resto.

Finalista 5 — "Névoa + missão primeiro, regra de acesso depois"

F2 + `mapHint.revealMission` mínimo, mas **sem** a regra rígida de acesso ao mapa (item 2 fica pra depois) — prioriza a recompensa narrativa (Narrative Writer) sobre a restrição de UX.

6. Votação

Agente	Voto	Justificativa / objeção
Game System Designer	#3	Registro fixo resolve o item 2 sem tocar em regra de D6; marcador mínimo já é suficiente pro item 3 agora.
Narrative Writer	#5 (dissenso)	Prioriza o desbloqueio por NPC (recompensa narrativa) sobre a restrição de acesso — toparia acesso rígido entrar depois sem perda real.
Frontend Engineer	#3	Névoa por nó de-risca o maior risco técnico novo; objeta a #4 — overlay de canvas sem precedente no mesmo ciclo que o resto é risco composto.
Backend Engineer	#3	Schema aditivo simples cobre os três dados (<code>discovered</code> , <code>known_markers</code> , <code>missions</code>) sem tabela nova.

AI Master Engineer	#2 (dissenso)	Quer o pacote mais barato possível enquanto o mecanismo de orçamento da Assembleia 04 não está pronto; objeta a #3/#4 pelo custo recorrente somado (SYSTEM_PROMPT maior, todo turno).
Infra Engineer	#3	Sem custo de infra em nenhum finalista; segue o consenso técnico.
Code QA Engineer	#3	Registro fixo bem definido, superfície de revisão menor que #4; proveniência do Kenney novo é o mesmo trabalho em qualquer finalista com arte nova.
Test Engineer	#3	Fatia testável pelo fluxo encadeado sem empilhar o risco do overlay visual (#4) no mesmo ciclo.
Realtime Multiplayer Engineer	#3	Sem posição forte; segue o consenso.

Resultado: Finalista 3 vence 6 a 2 a 1 (AI Master Engineer dissente pra #2; Narrative Writer dissente pra #5).

7. Vencedor — com as mitigações da minoria incorporadas

Finalista 3, "Arte + névoa por nó + regra de acesso + marcador de missão mínimo", com dois ajustes vindos dos dissensos:

- Do dissenso do AI Master Engineer (#2)** — incorporado como sequência, não como corte: o pacote de código (registro de local + remoteArea + revealMission) entra nesta fase, **mas o mesmo portão de lançamento já em vigor desde a Assembleia 04 continua valendo** — nenhum convite novo de teste até o mecanismo de proteção de orçamento estar pronto, agora protegendo três acréscimos de custo empilhados (semente de campanha, mapas avançados, e o que mais vier), não só dois.
- Do dissenso do Narrative Writer (#5)** — já estava dentro do Finalista 3 (marcador de missão mínimo incluso); reforço que o número de locais distantes com mapa próprio desenhado começa **pequeno** (1, talvez 2) nesta fase, não um catálogo aberto — dá pro Narrative Writer manter a curadoria real que ele pediu.

O que entra na v0.7 (proposta de corte)

- Arte melhorada: exteriores com terreno gerado por código (textura + caminhos em curva) + Kenney Roguelike/RPG pack pra cenário/decoração; interiores com mobília real do mesmo pacote (mesa, cadeira, escada, balcão — a verificar visualmente antes de produção, Seção 1.1.1).
- Névoa de descoberta **por nó** — `discovered/known_markers` persistidos por campanha; local não visitado não aparece, marcador de missão conhecido aparece mesmo sem visita.
- Regra de acesso ao mapa — registro fixo de tipo de local (cidade/masmorra/missão distante) + `mapHint.remoteArea` pro que é narrado ad-hoc; mapa bloqueado fora de área tipo cidade.
- Marcador de missão mínimo — `mapHint.revealMission`, sem sistema de missão completo, sem catálogo pré-escrito.
- 1 (talvez 2) local(is) distante(s) novo(s), curados de verdade pelo Narrative Writer — não um número aberto.

O que fica pra depois (v0.8+), e por quê

- **Overlay visual real de escuridão** (o "mapa fica escuro" literal) — maior risco técnico novo (canvas sobre Leaflet, sem precedente), fatiado pra depois de validar que o dado de descoberta está sólido com a versão mais simples primeiro. **Gap real reconhecido:** a v0.7 não entrega o efeito visual de névoa que o Tiago descreveu literalmente — entrega o dado e a regra por trás dela, com a renderização mais simples por enquanto.
- **Sistema de missão completo** (objetivos, condição de conclusão, recompensa, jornal/diário) — feature própria do tamanho de uma assembleia inteira, não cabe aqui.
- **Camada de mapa de mundo/região** (um "menu de viagem" acima da cidade, se locais distantes se multiplicarem) — só vira necessária se o número de locais distantes crescer além do que um `remoteArea` simples aguenta explicar.
- Mais locais distantes além do primeiro — cada um exige curadoria real do Narrative Writer, não é trabalho só de código.

Aguardando aprovação do Tiago

Nenhuma linha de código deste plano até o sinal explícito dele — só este documento não depende de aprovação, é processo, não produto.